

文件与数据流

主讲老师：高祥



2024/12/12

Xiang Gao



北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

期末考试

- 闭卷考试，上机考试
- 2025年1月7日下午13:20-15:20，提前30分钟进入考场
- 单选题(30)、多选题(5)、编程题&简答题(4)
- 主要考试内容
 - 面向对象核心思想 (e.g., 设计原则)
 - 封装 (e.g., 目的、访问范围、重载、静态属性.....)
 - 继承 (继承的优缺点、重写/覆盖、转型、super.....)
 - 多态 (e.g., 多态的必要条件、表现形式.....)
 - 设计原则&设计模式
 - 其他内容：集合、异常、多线程、网络、I/O (各有1-2道选择题)

主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

问答题

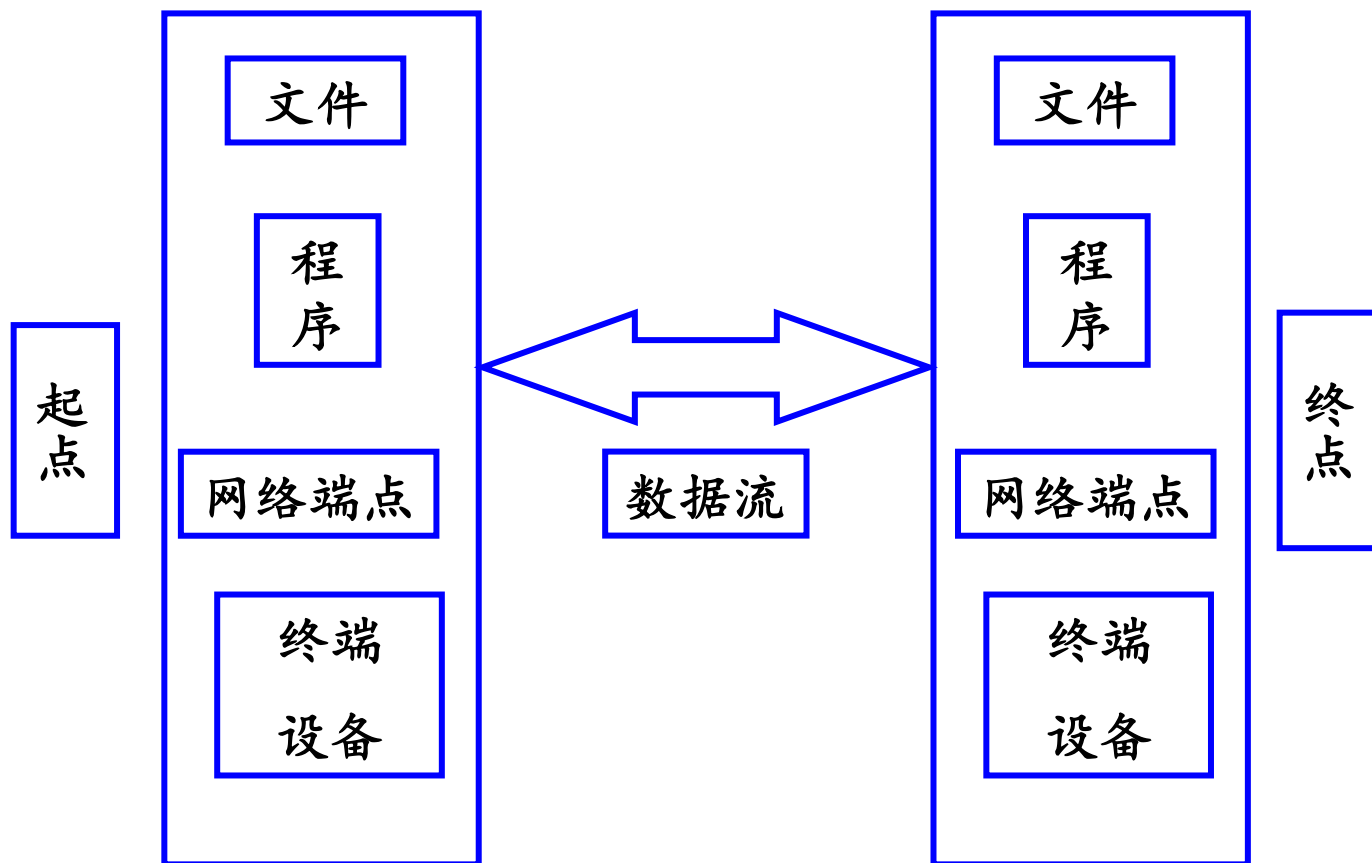
- 如何接受命令行输入？
- 如何接受键盘输入（标准输入）？
- 如何知道文件系统中有哪些目录和子目录，如何知道文件的属性？例如可读，可写等
- 如何读写文件？（字节文件，字符文件，顺序读，随机读，对象的序列化和重构等）

什么是输入输出流？

- ✓ **输入输出流**：说的都是数据流，而**数据流**是指一组有顺序的、有起点和终点的数据集合。
- ✓ Java的输入输出是由定义在Java.io这个包中的一系列类和接口来实现的，所以在需要输入输出的程序的前面都要加上

import java.io.*;

什么是输入输出流？

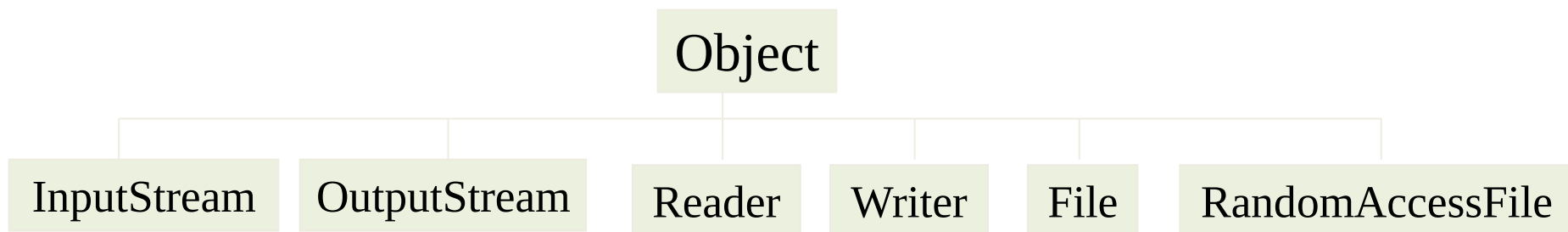


什么是输入输出流？

- ✓ Java程序不能直接操纵I/O设备，而是在程序和设备之间加入了一个中间介质，这就是流。
- ✓ 建立流实际上就是建立数据传输通道，将起点和终点连接起来。
- ✓ 流是数据传输的抽象表达，与具体设备无关。程序一旦建立了流，就可以不用理会起点或终点是何种设备。

什么是输入输出流？

- ✓ 输入流将数据从文件、标准输入或其它外部设备输入加载到内存；
- ✓ 输出流的作用正好相反，将内存中的数据保存到文件中，或传输给输出设备。



什么是输入输出流？

- 输入数据流：输入数据流只能读, 不能写
 - 字节流：Java中的输入数据流(字节流)都是抽象类InputStream的子类；
 - 字符流：Java中的输入数据流(字符流)都是抽象类Reader的子类；
- 输出数据流：输出数据流只能写, 不能读
 - 字节流：java中的输出数据流(字节流)都是抽象类OutputStream的子类；
 - 字符流：java中的输出数据流(字符流)都是抽象类Writer的子类；

主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

如何接受命令行输入? Command-Line Arguments

- These string arguments are placed on the command line to launch the Java interpreter, after the class name:

```
java TestArgs arg1 arg2 "another arg"
```

- Each command-line argument is placed in the args array that is passed to the static main method:

```
public static void main(String[] args)
```

如何接受命令行输入? Command-Line Arguments

```
public class CommandLine {  
    Run | Debug  
    public static void main(String[] args) {  
        for(int i = 0; i<args.length; i++)  
            System.out.println("args[" + i + "] = " + args[i]);  
    }  
}
```

```
$ java cn.edu.buaa.oop.lesson_12.CommandLine test1 test2 test3  
args[0] = test1  
args[1] = test2  
args[2] = test3
```

主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

- The variable **System.out** enables you to write to *standard output*. It is an object of type **PrintStream**.
- The variable **System.in** enables you to read from *standard input*. It is an object of type **InputStream**.
- The variable **System.err** enables you to write to *standard error*. It is an object of type **PrintStream**.

- The **println** methods print the argument and a newline character (`\n`).
- The **print** methods print the argument without a newline character.
- The **print** and **println** methods are overloaded for most primitive types (**boolean, char, int, long, float, and double**) and for **char[], Object, and String**.
- The **print(Object)** and **println(Object)** methods call the **toString** method on the argument.

标准输入 Reading From Standard Input

```
public static void main(String[] args) {  
    String s;  
    InputStreamReader ir = new InputStreamReader(System.in);  
    BufferedReader in = new BufferedReader(ir);  
  
    try{  
        System.out.print(s: "Input is : " );  
        s = in.readLine();  
        while (s!=null && !s.equals(anObject: "end")){  
            System.out.println("Output is : " + s);  
  
            System.out.print(s: "Input is : " );  
            s = in.readLine();  
        }  
        in.close();  
    } catch(IOException e){  
        e.printStackTrace();  
    }  
}
```



Simple Formatted Output

- You can use the formatting functionality as follows

```
out.printf("name count\n");
```

```
String s = String.format("%s %5d%n", user, total);
```

Java 中的String.format使用

转 换 符	说 明	示 例
%s	字符串类型	"mingrisoft"
%c	字符类型	'm'
%b	布尔类型	true
%d	整数类型（十进制）	99
%X	整数类型（十六进制）	FF
%o	整数类型（八进制）	77
%f	浮点类型	99.99
%a	十六进制浮点类型	FF.35AE
%e	指数类型	9.38e+5
%g	通用浮点类型（f和e类型中较短的）	
%h	散列码	
%%	百分比类型	%
%n	换行符	
%tx	日期与时间类型（x代表不同的日期与时间转换符	

Java 中的String.format使用

标 志	说 明	示 例	结 果
+	为正数或者负数添加符号	(""+d",15)	+15
-	左对齐	("%-5d",15)	15
0	数字前面补0	("%04d", 99)	0099
空格	在整数之前添加指定数量的空格	("% 4d", 99)	99
,	以","对数字分组	("%,f", 9999.99)	9,999.990000
(	使用括号包含负数	("%(f", -99.99)	(99.990000)
#	如果是浮点数则包含小数点，如果是16进制或8进制则添加0x或0	(""%#x", 99) ("%#o", 99)	0x63 0143
<	格式化前一个转换符所描述的参数	(""%f和"%<3.2f", 99.45)	99.450000和99.45
\$	被格式化的参数索引	(""%1\$d,%2\$s", 99,"abc")	99,abc

Scanner 类的使用

- Scanner的作用：通过分隔符模式将输入分解为标记，默认情况下该分隔符模式与空白匹配。
- 然后可以使用不同的 next 方法将得到的标记转换为不同类型的值。
- 通过 Scanner 类的 next() 与 nextLine() 方法获取输入的字符串，在读取前我们一般需要使用 hasNext 与 hasNextLine 判断是否还有输入的数据
- **A Scanner can be used with console input streams as well as file or network streams.**

主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

文件输入输出

- 在对一个文件进行输入输出(I/O)操作之前,必须先获得有关这个文件的基本信息,例如
 - 文件的路径(包括绝对路径和相对路径)和文件名
 - 以及这个文件有没有被读取或者被写入的权限
 - 有时还想知道文件的长度是多少、是什么时候创建的等等

File 类

- File 类代表与平台无关的文件和目录
- File 能新建、删除、重命名文件和目录
- 类File一般不涉及文件内部的具体内容，而是从整体上对文件进行处理，因此类File不能访问文件内容本身。如果需要访问文件内容本身，则需要使用文件输入/输出流（FileInputStream/FileOutputStream）

File 类

- 访问文件名：

- getName()
- getPath()
- getAbsolutePath()
- getAbsolutePath()
- getParent()
- renameTo(File newName)

- 文件检测：

- exists()
- canWrite()
- canRead()
- isFile()
- isDirectory()
- 获取常规文件信息
 - lastModify()
 - Length()

- 文件操作相关

- createNewFile()
- delete()
- 目录操作相关
 - mkdir()
 - list()
 - listFiles()

动态生成目录和文件: CreateFile.java

```
import java.io.*;
public class CreateFile{
    private static File f1, f2, f3, p1, p2;
    public static void main(String[] s){
        f1 = new File("aNewFile.txt");
        p1 = new File("bdfz2");
        f2 = new File(p1, "F.sql");
        p2 = new File("bdfz/root/");
        f3 = new File("bdfz/root/", "anotherFile.java");
        try{
            f1.createNewFile();
            p1.mkdir();
            f2.createNewFile();
            p2.mkdirs();
            f3.createNewFile();
        }catch(IOException ioe){}
    }
}
```

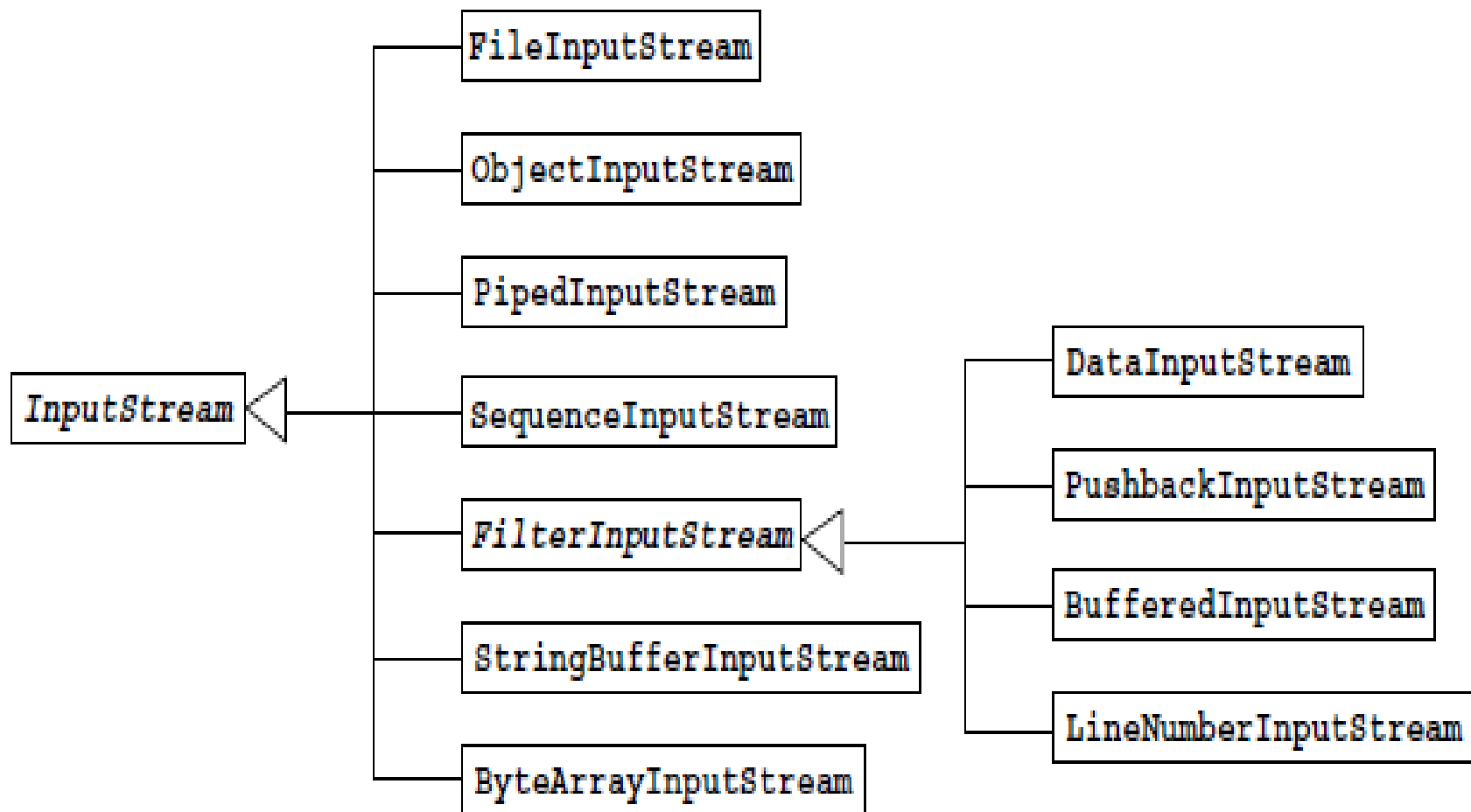
File(文件类)

- 在Java文件体系中,文件和目录是统一抽象成文件来处理的,不过如果一个“File”是目录的话,它就另外具有一种方法——list(),用这个方法来得取得文件名列表以及子目录结构等。

主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

字节流: InputStream Class Hierarchy



The InputStream Class Hierarchy

- `FileInputStream`: 用于从本地文件中读出数据。
- `PipedInputStream`: 用于管道输入/输出时从管道中读取数据
- `BufferedInputStream` 读取数据时可以对数据进行缓冲，这样可以提高效率和增加特殊功能。
- `DataInputStream` 实现了 `java.io` 包中的 `DataInput` 接口，读取数据的同时，可以对数据进行格式处理。因此，能用来读取 `java` 中的基本数据类型。

The InputStream Class Hierarchy

- `ByteArrayInputStream`: 包含一个内存缓冲区，用于从内存中读取数据。
- `SequencedInputStream`: 用来把两个或更多的`InputStream`输入流对象转换为单个`InputStream`输入流对象使用。
- `ObjectInputStream`: 用来读取对象
- `AudioInputStream`: `Audio`的输入输出

InputStream

- InputStream是抽象类，所以不能通过“new InputStream()”的方法构造InputStream的实例。但它声明了输入流的基本操作，包括读取数据(*read*)、关闭输入流(*close*)、获取流中可读的字节数(*available*)、移动读取指针(*skip*)、标记流中的位置(*mark*)和重置读取指针(*reset*)等，它的子类一般都重写这些方法。
- 通过构造InputStream子类的实例方式可以获得InputStream类型的实例。

The InputStream Methods

- ✓ 方法 `read()` 提供了三种从流中读数据的方法。
 - ✓ `int read()`: 一次只能读一个字节, 抽象方法。
 - ✓ `int read(byte b[])`: 一次读多个字节到数组中
 - ✓ `int read(byte[], int off, int len)`;
- ✓ 一般 `available` 和 `read()` 混合使用, 这样在读操作前可以知道有多少字符需要读入。
- ✓ `mark` 通常与 `reset()` 方法配合使用, 可重复读取输入流所指定的字节数据。

The InputStream Methods

- 例如：若fis是一个输入流，如果想读取相同的字节数据到字节数组b1和b2中，可通过以下语句实现，先标记输入流中当前位置，读区数据后重置读取位置。
 - `fis.mark(5);`
 - `byte[] b1=new byte[5], byte[] b2=new byte[5],`
 - `fis.read(b1, 0, 5);`
 - `fis.reset();`
 - `Fis.read(b2, 0, 5);`

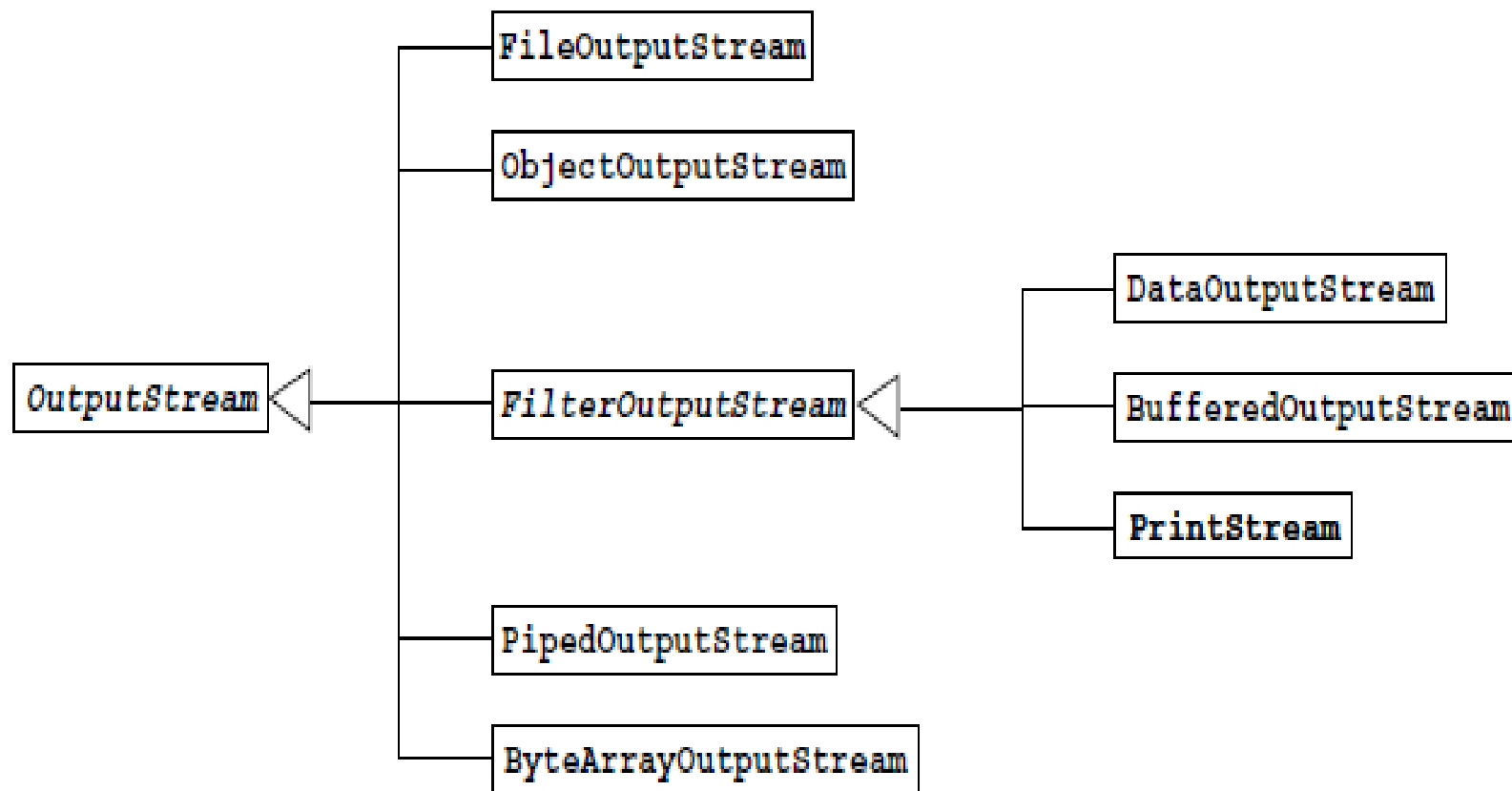
FileInputStream

- FileInputStream是InputStream的子类，用于从本地文件中读取数据，他有三个构造方法。
 - 对文件内容进行操作的基本步骤如下：
 - 创建该文件所对应的实例，以获得相关的资源，如存放该文件信息的内存空间和对该文件的控制权限。
 - 对该文件进行读（输入）/写（输出）操作。
 - 最后关闭该文件，以释放所占用的资源。

示例：读文件，一次读一个字节

```
public class J_Example {  
    public static void main(String[] args) {  
        int i;  
        int b;  
        try {  
            FileInputStream f = new FileInputStream(  
                "F:\\BaiduYunDownload\\eclipse-jee-kepler-SR1-win32-x86_64\\eclipse\\"  
                + "workspace\\ioExample\\src\\com\\dal\\ioExample\\test.txt");  
  
            b = f.read();  
            for (i = 0; b != -1; i++) {  
                System.out.print((char) b);  
                b = f.read();  
            }  
            System.out.println();  
            System.out.println("TOTAL=" + i);  
            f.close();  
        } catch (FileNotFoundException e) {  
            System.out.println(e.toString());  
        } catch (IOException e) {  
            System.out.println(e.toString());  
        }  
    }  
}
```

字节流: OutputStream Class Hierarchy



The OutputStream Class Hierarchy

- `FileOutputStream`: 用于向本地文件中写入数据。
- `PipedOutputStream`: 用于管道输入/输出时把数据向管道输出
- `DataOutputStream`: 提供了对java的基本数据类型的支持
- `PrintStream`: 提供了向屏幕输出有格式数据的很多方法

OutputStream

- OutputStream是抽象类，所以不能通过“new OutputStream()”的方法构造OutputStream的实例。但它声明了输出流的基本操作，包括输出数据(*write*)、关闭输出流(*close*)、清空缓冲区(*flush*)等。

FileOutputStream

- `FileOutputStream`是`OutputStream`的子类，他有五个构造方法；其中，`FileOutputStream(String name)`；和`FileOutputStream(String name, boolean append)`；这两个构造方法最常用。
- 应用`FileOutputStream`将数据写入文件的基本步骤如下：
 - 创建`FileOutputStream`的实例，以获得相关的资源；
 - 将数据写入文件中；
 - 最后关闭文件，以释放所获得的资源。

示例: J_Example2.java

```
import java.io.OutputStream;
import java.io.IOException;

public class J_Example2 {
    public static void main(String[] args) {
        String s = "Wonderful OutputStream";
        byte[] b;
        OutputStream out = System.out;
        b = s.getBytes();
        try {
            out.write(b);
            out.flush();
        } catch (IOException e) {
            System.err.println(e);
        }
    }
}
```


✓ **FileInputStream(String name)**

throws FileNotFoundException

✓ 例如：读取C盘javatemp目录下的文件test.txt

FileInputStream fis=

new FileInputStream(“c:\\javatemp\\test.txt”)

✓ **FileInputStream(File f)**

throws FileNotFoundException

✓ 例如：

File file=new File(“./temp/test.txt ”);

FileInputStream fis=new FileInputStream(file);

- ✓ **FileOutputStream(String name)**
throws FileNotFoundException
- ✓ **FileOutputStream(String name, boolean append)**
throws FileNotFoundException

主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

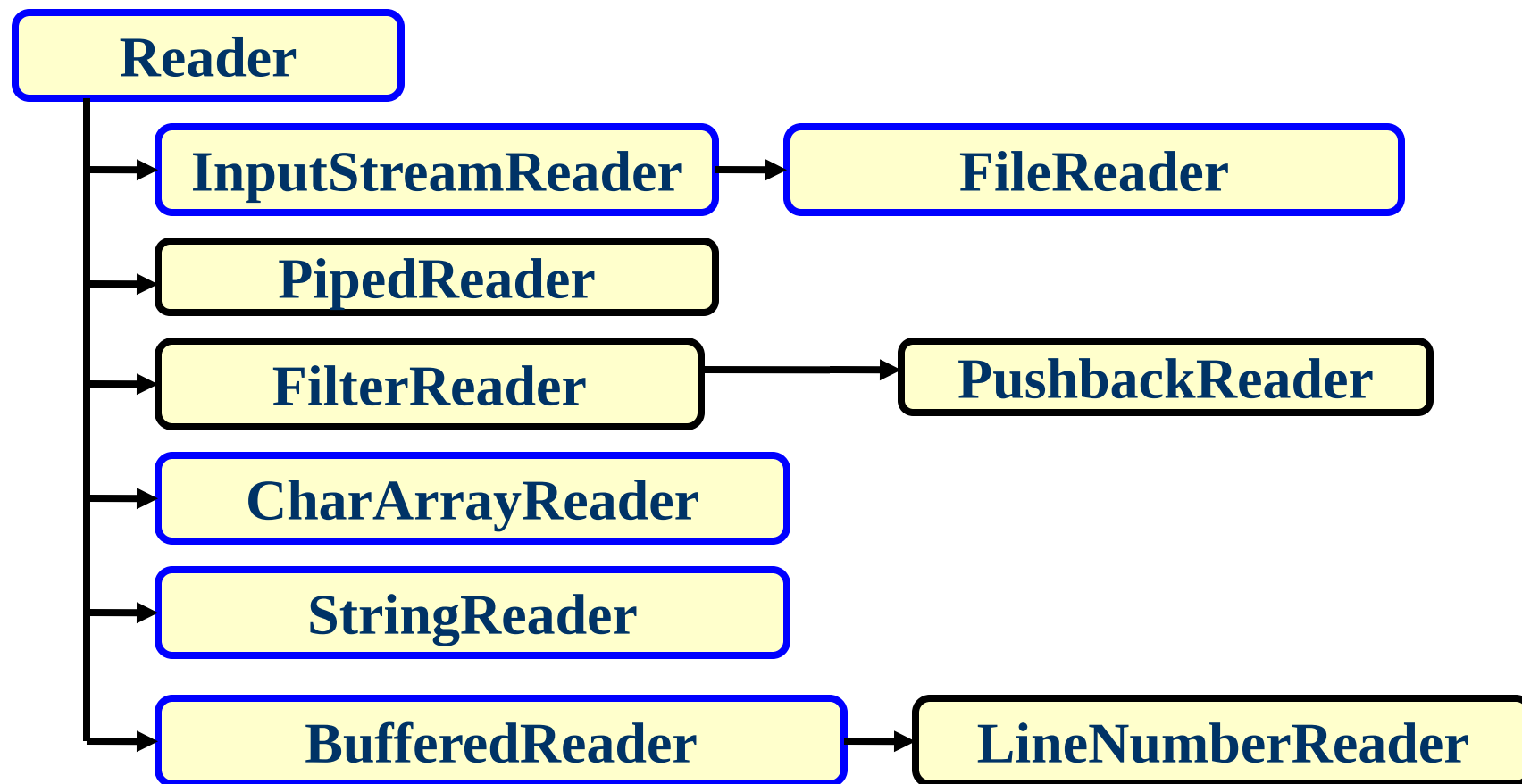
输入数据流reader (字符流)的继承关系

- Reader类和Writer类派生的子类与输入流(InputStream)和输出流(OutputStream)的子类几乎是完全对应的,只不过它们处理的是字符流而不是字节流。
- 如果程序读到的数据是不同国家的语言,其编码不同,那么程序应使用Reader和Writer流。
- Reader类和Writer类中的大部分方法与InputStream类和OutputStream类中的对应方法名称相同,只是读取或写入的数据是字符、字符数组和字符串等。

输入数据流reader (字符流)的继承关系

- Reader和Writer是以字符为基本单位访问文件的，读写器处理的是字符流；
- Reader和Writer也是抽象类，对应了读写器的一些基本操作。Reader用来读取数据，Writer用来存储数据。
- FileReader和FileWriter分别是reader和Writer的子类；

输入数据流reader (字符流) 的继承关系



InputStreamReader类和OutputStreamWriter类

- InputStreamReader类继承自Reader类，通过其read方法从字节输入流中读取一个或多个字节数据转换为字符数据，它不是一个缓冲流，因此其转换的效率并不高。
- OutputStreamWriter类继承自Writer类，其作用是转变字符输出流为字节流输出。

InputStreamReader 实现了字节到字符的转变

- InputStreamReader 是字节流与字符流之间的一个桥梁，实现了字节到字符的转变；
- 为了提高效率常常把它与BufferedReader一起用：
 - 例：

```
BufferedReader in = new BufferedReader (new  
    InputStreamReader (System.in));
```


示例

```
import java.io.*; //(从键盘上读一个字符串)
public class CharInput
{ public static void main(String args[])
    throws IOException
    {
        InputStreamReader ir;
        BufferedReader in;
        ir=new InputStreamReader(System.in);
        in=new BufferedReader(ir);
        String s=in.readLine();
        int i=Integer.parseInt(s); i=i*2;
        System.out.println("the result is " +i);
    }
}
```

OutputStreamWriter 实现了字符到字节的转换

- OutputStreamWriter 是字符流与字节流之间的一个桥梁，实现了字符到字节的转变；
- 为了提高效率常常把它与 BufferedWriter 一起用，
— 例：

```
Writer out = new BufferedWriter(  
    new OutputStreamWriter(System.out));
```

java 中的unicode

- 问题:
- 如果字符流不是来自本地,有可能编码不一样,直接读取会读出不正确字符
- 处理方法:
 - `ir=new InputStreamReader(is, “utf-8” );`

字符文件输入/输出流FileReader类和FileWriter类

- **FileReader和FileWriter类分别是Reader和Writer子类，他们分别用来从字符文件读取字符和向字符文件输出字符数据。**
- **FileReader等价于FileInputStream类，FileWriter类等价于FileOutputStream类，其区别在于一个是字符流，一个是处理字节流。**

注意事项

✓ `FileReader` 类是 `InputStreamReader` 的子类，`FileWriter` 类是 `OutputStreamWriter` 的子类，他们可以方便地用来处理字符文件，自己本身没有更进一步的扩展，方法都是从其超类继承来的；

✓ 如果想处理字节文件，建议使用 `FileOutputStream` 或者 `FileInputStream`

示例:

- 将一个文件复制到另一个文件中，使用基于Unicode字符的文件读写器

```
import java.io.*;
public class Copy {
    public static void main(String [] args)
        throws IOException {
        FileReader in=new FileReader("Copy.java");
        FileWriter out=new FileWriter("Copy.txt");
        int c;
        while ((c=in.read( ))!= -1)
            out.write(c);
        in.close( );
        out.close( );
    }}
}
```

示例:

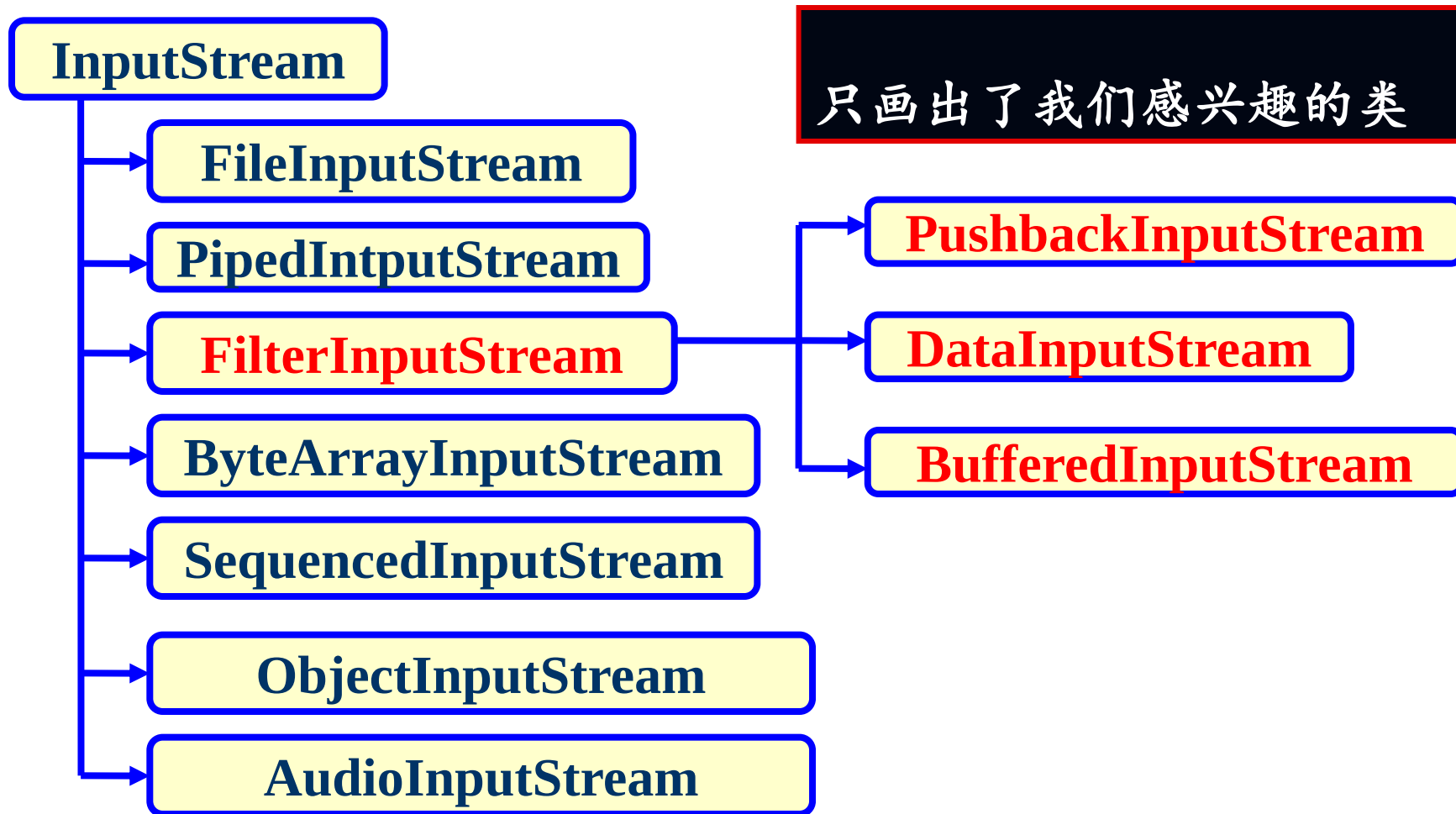
- 将一个文件复制到另一个文件中，使用基于字节的文件输入/输出流。

```
import java.io.*;
public class BytesCopy {
    public static void main(String [] args)
        throws IOException {
        FileInputStream in=
            new FileInputStream("BytesCopy.java");
        FileOutputStream out=
            new FileOutputStream("BytesCopy.txt");
        int c;
        while ((c=in.read( ))!= -1)
            out.write(c);
        in.close( );}}
```

主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

缓冲流、数据流、回压流和打印流



过滤流

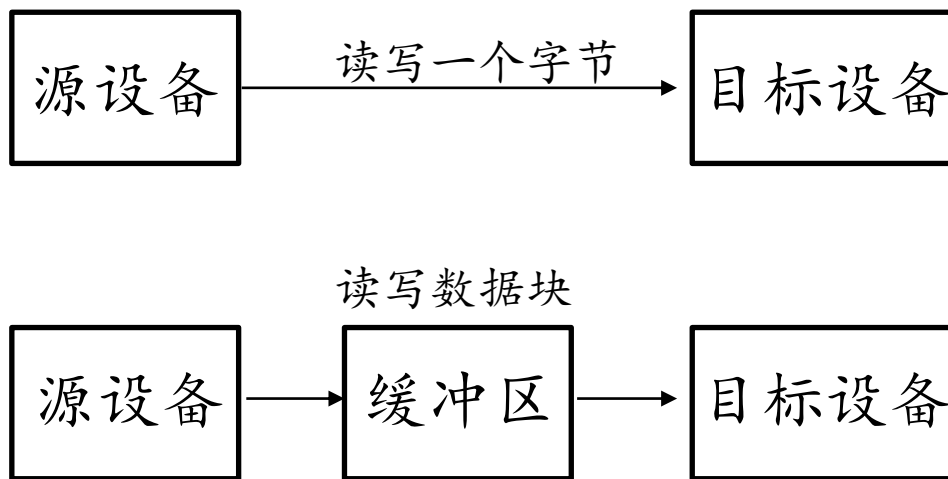
- `FilterInputStream`和`FilterOutputStream`是两个抽象类，都属于过滤流类。
- 过滤流类的主要功能是为输入/输出流提供一个通用的接口，提供将流连接在一起的能力，即将一个流的输出连接到另一个流的输入，这样可以得到满足应用程序要求的很长的输入和输出过滤器。

主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

带缓存的输入流和输出流

- 借助于字节数组缓存，在读取或存储数据时可以将一个较大数据块读如内存中，或将内存中较大数据块一次性写入指定的文件中，从而达到提高读/写效率的目的。



带缓存的输入流和输出流

- 带缓存的输入流和输出流对应的类是：

BufferedInputStream和BufferedOutputStream：

- 当创建BufferedInputStream或BufferedOutputStream的实例时，均会在内存中开辟一个字节数组的存储单元（一般称为缓存），默认为512字节，用来存放数据流中的数据。
- 他们用来连接别的输入/输出流，以获得读写性能上的提高。
- 这两个类通过缓存机制进一步增强了输入流（InputStream）和输出流（OutputStream）读取和存储数据的效率。

带缓存的输入流和输出流

- ✓ 方法flush(): 当通过BufferedOutputStream写数据时, 数据不直接写入数据流, 而是先写入缓冲区, 当缓冲区满时, 数据才会写入所连接的数据流。当缓冲区未滿时, 可以用该类的方法flush(), 将缓冲区的数据强制全部写入输出流。
- ✓ 第一次读取数据按块读入缓冲区, 此后如果有数据读取操作则直接访问缓冲区。缓冲区的作用除了可以提高性能外, 还提供了标记读取位置(mark方法)和重置标记(reset方法)的支持。

JFileInputStream.java

```
public class J_FileInputStream{
    public static void main(String[] args){
        InputStream f;
        int ch;
        long i=0;
        System.out.println("Start!");
        Date d1=new Date();
        try{
            f=new FileInputStream("F:\\BaiduYunDownload\\eclipse-jee-kepler-SR1-w
            while((ch=f.read())!=-1)
                i++;
            f.close();
        }
        catch(Exception e){
            System.out.println(e);
            System.exit(-1);
        }
        Date d2=new Date();
        long elapsedTime=d2.getTime()-d1.getTime();
        System.out.println("Stop!");
        System.out.println("Read a"+i+"-byte file");
        System.out.println("using" + elapsedTime+"Milliseconds(0.001 Seconds).");
    }
}
```

JBufferedInputStream.java

```
public class J_BufferedInputStream{
    public static void main(String[] args){
        InputStream f;
        int ch;
        long i=0;
        System.out.println("Start!");
        Date d1=new Date();
        try{
            f=new BufferedInputStream(new FileInputStream("F:\\BaiduYunDownload\\"));
            while((ch=f.read())!=-1){
                i++;
            }
            f.close();
        }
        catch(Exception e){
            System.out.println(e);
            System.exit(-1);
        }
        Date d2=new Date();
        long elapsedTime=d2.getTime()-d1.getTime();
        System.out.println("Stop!");
        System.out.println("Read a"+i+"-byte file");
        System.out.println("using" + elapsedTime+"Milliseconds(0.001 Seconds).");
    }
}
```


示例：输出结果

```
D:\java>java J_BufferedInputStream test.txt
Start!
Stop!
Read a11132928-byte file
using594Milliseconds(0.001 Seconds).
```

```
D:\java>java J_FileInputStream test.txt
Start!
Stop!
Read a11132928-byte file
using27844Milliseconds(0.001 Seconds).
```

主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

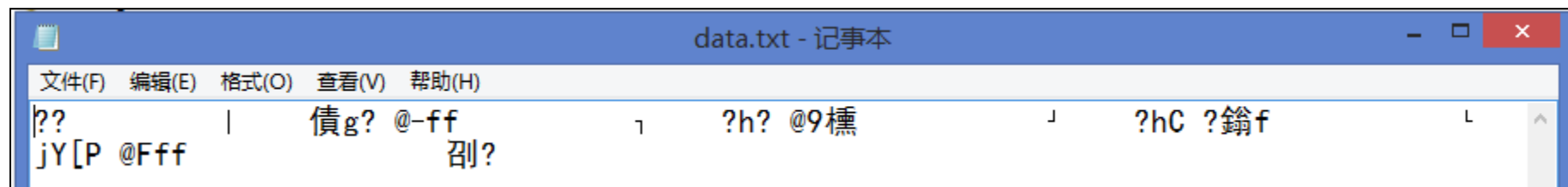
DataInputStream与DataOutputStream

- **类DataInputStream与DataOutputStream:**
主要用来读取于存储基本数据类型的数据。而且数据流的存储格式采用统一的形式，即与具体的计算机无关。
- 一般来说，DataInputStream是用来读取DataOutputStream存储的数据。
- DataInputStream与DataOutputStream的构造方法分别要求以输入流与输出流的实例引用作为参数。

示例: DataTest.java

```
public class DataTest {  
    public static void main(String[] args) throws IOException {  
        FileOutputStream fout = new FileOutputStream("data.txt");  
        DataOutputStream out = new DataOutputStream(fout);  
        float[] prices = { 1.5f, 2.1f, 2.9f, 1.8f, 3.1f };  
        int[] units = { 5, 2, 4, 3, 1 };  
        String[] items = { "苹果", "鸭梨", "蜜桃", "橙子", "葡萄" };  
        for (int i = 0; i < prices.length; i++) {  
            out.writeFloat(prices[i]);  
            out.writeChar('\t');  
            out.writeInt(units[i]);  
            out.writeChar('\t');  
            out.writeChars(items[i]);  
            out.writeChar('\n');  
        }  
        out.close();  
        fout.close();  
    }  
}
```

示例: DataTest.java (续)



```
FileInputStream fin = new FileInputStream("data.txt");
DataInputStream in = new DataInputStream(fin);
float price, total = 0.0f;
int unit;
char ch;
String item;
```

示例: DataTest.java (续)

```
try {
    while (true) {
        price = in.readFloat();
        in.readChar();// 跳过Tab
        unit = in.readInt();
        in.skip(2);// 跳过Tab
        item = new String();// 清空字符串
        while ((ch = in.readChar()) != '\n')
            // 每次读一个字符, 碰到换行符结束
            item += ch;// 将字符合成字符串
        System.out.println
            ("你买了" + unit + "斤" + item + " 单价是" + price);
        total += unit * price;
    }
} catch (EOFException e) {
    // 读到文件尾将抛出异常并终止循环
    in.close();
    fin.close();
    System.out.println("你总共花去" + total + "元");
}
```

示例: DataTest.java (续)

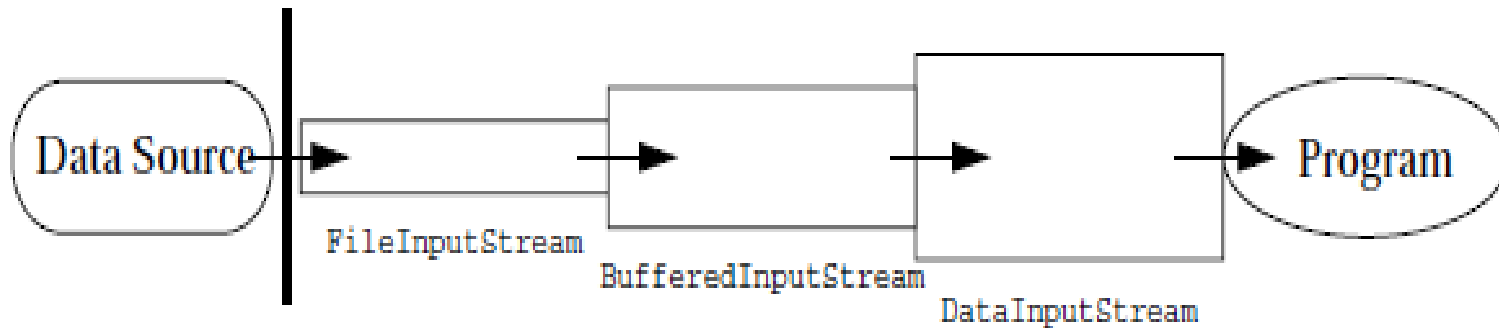
你买了 5 斤苹果 单价是 1.5
你买了 2 斤鸭梨 单价是 2.1
你买了 4 斤蜜桃 单价是 2.9
你买了 3 斤橙子 单价是 1.8
你买了 1 斤葡萄 单价是 3.1
你总共花去 31.8 元

示例：DataTest.java (续)

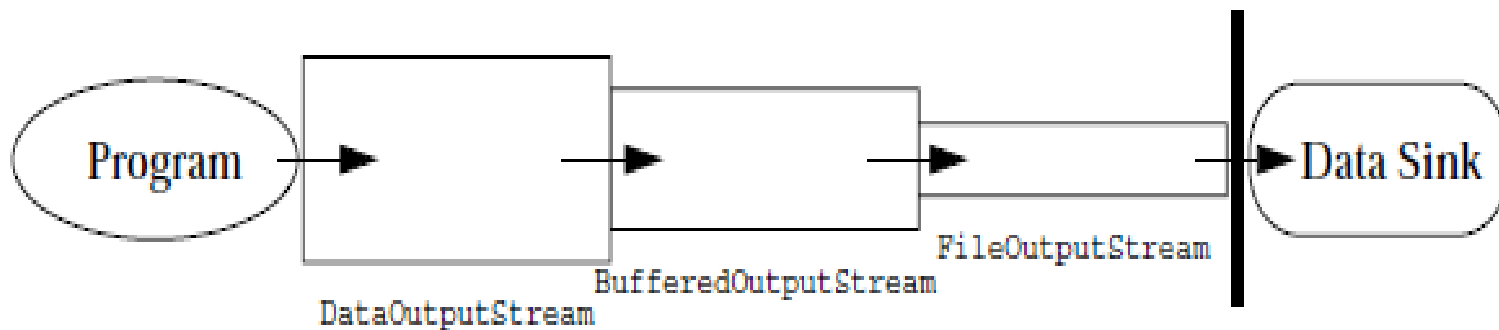
- 程序采用了一个条件恒为真的循环读取整个文件内容，那么怎样才能退出循环呢？
 - 显然不能用读取某个数据为退出条件，前面例子中有读取一个-1时结束循环的用法，这个方法在此处无效，因为文件中的数据很可能就包含-1。
 - 文件内容读完后再次试图读取，系统就会抛出一个 **EoFException** 异常，我们就用它来结束循环。

I/O Stream Chaining

Input Stream Chain



Output Stream Chain



主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

对象流

- **必要性**：有时我们希望把程序中创建的一些类对象保存到文件中，以供其他的程序使用。
- 对于一个类对象，**它的属性本身就包含各种类型变量或其他类对象**，那么如何实现对象的读/写呢？
- Java.io包中提供了**对象流ObjectInputStream类和ObjectOutputStream类**，**用来创建对象输入流和对象输出流**，读取文件中的对象或将对象写入文件中。

对象流

- **创建对象流：**
 - `Public ObjectOutputStream(InputStream in);`
 - `Public ObjectOutputStream(OutputStream in);`
- **读写对象流：**
 - `Public final void writeObject(Object obj)`
 - `throws IOException`
 - *Public final void readObject()*
 - *Throws OptionalDataException,*
 - *ClassNotFoundException,IOException*

- 需要注意的是，使用对象流读取和写入对象时，要保证对象是**串行化(Serializable)**的；
- 串行化的概念：*指对象通过把自己转化为一系列字节，记录字节的状态数据，以便再次利用的这个过程。*

串行化(Serializable)

- 对象串行化方法的一个典型应用是编写网络应用程序，通过对象串行化方法可以在另一台主机中重构指定的对象。
- 一个类要具有可串行化的特性就必须实现接口 `java.io.Serializable`。对于可以串行化的对象可以用 `java.io.ObjectOutputStream` 来输出该对象，用 `java.io.ObjectInputStream` 来读入该对象。

示例:

```
//write a Serializable object into a file

import java.io.*;
class J_Student implements Serializable{
    String m_name;
    int m_id;
    int m_height;
    public void mb_output(){
        System.out.println("Name =" + m_name);
        System.out.println("ID =" + m_id);
        System.out.println("Height =" + m_height);
    }
}
```

示例:

```
//write a Serializable object into a file

import java.io.*;
public class J_ObjectOutputStream{
    public static void main(String[] args)throws Exception{
        ObjectOutputStream f=
            new ObjectOutputStream(new FileOutputStream("object.dat"));
        J_Student s=new J_Student();
        s.m_name="Zhang, San";
        s.m_id=2003001;
        s.m_height=172;
        f.writeObject(s);
        s.mb_output();
        f.close();
    }
}
```


示例:

```
//Read a Serializable object from file

import java.io.*;

public class J_ObjectInputStream{
    public static void main(String[] args) throws Exception{
        ObjectInputStream f=
        new ObjectInputStream(new FileInputStream("object.dat"));
        J_Student s=(J_Student)(f.readObject());
        s.mb_output();
        f.close();
    }
}
```

主要内容

- Java输入输出
 - 命令行输入
 - 控制台读入数据、标准输入输出
- Java文件管理（File类）
- 字节输入输出流
- 字符输入输出流
- 过滤流
 - 缓冲流
 - 数据流
 - 对象流
 - 管道流

管道流

- 与前面讲过的其他流类不同的是管道数据流的这两个类一定是成对的，同时使用并相互连接的，这样才形成一个数据通信管道。 管道数据流主要用于线程间的通信。
- 创建管道输入流(PipeInputStream)对象
 - Public PipeInputStream();
 - Public PipeInputStream(PipeOutputStream src);
- 创建管道输出流(PipeOutputStream)对象
 - Public PipeOutputStream();
 - Public PipeOutputStream(PipeInputStream src);

- 管道连接方法（对没有连接的管道流）：
 - *Public void connect(PipeInputStream src)*
 - *Public void connect(PipeOutputStream src)*
- 管道输入流中还增加了一个方法：
 - *Protected void receive(int b) throws IOException //*
用来接收一个字节数据b,如果管道中没有数据,
该方法将阻塞。

使用管道流

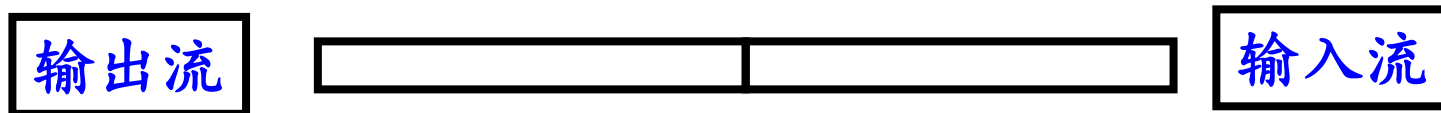
- **创建管道流:**
- `PipedInputStream pis=new PipedInputStream();`
- `PipedOutputStream pos=new PipedOutputStream(pis);`
- **或:**
- `PipedOutputStream pos=new PipedOutputStream();`
- `PipedInputStream pis=new PipedInputStream(pos);`
- **或**
- `PipedInputStream pis=new PipedInputStream();`
- `PipedOutputStream pos=new PipedOutputStream();`
- `Pis.connect(pos);`

注意事项

- 被连接的管道流必须没有与任何别的管道流连接，否则会抛出 `IOException` 异常，管道输出流输出数据到管道，管道输入流在管道中读取数据
- 因为管道流是 `InputStream` 和 `OutputStream` 类的子类，因此它能作为别的输入/输出流的参数，即连接到别的流，从而增加其输入/输出的功能。

例如：

- 下面的代码创建了数据输出流对象dos，并与管道输出流out连接，这样，通过dos能向管道中输入各种类型的数据，而不仅仅是字节数据。
- `PipedOutputStream out=new PipedOutputStream();`
- `Dos=new DataOutputStream(out);`



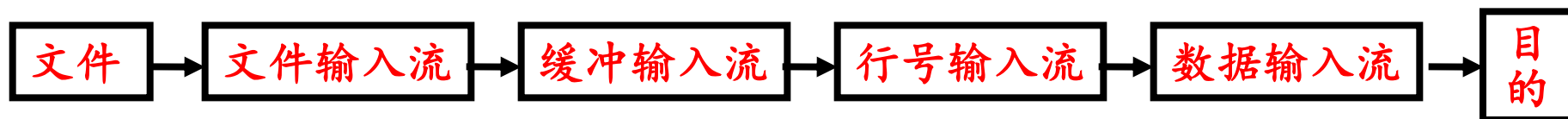
管道流一定是输入输出并用

示例:

```
import java.io.*;
class pipedstream
{
    public static void main(String args[]) throws IOException
    {
        byte aByteData1=123, aByteData2=111;
        PipedInputStream pis=new PipedInputStream();
        PipedOutputStream pos=new PipedOutputStream(pis);
        System.out.println("PipedInputStream");
        try{
            pos.write(aByteData1);
            pos.write(aByteData2);
            System.out.println((byte)pis.read());
            System.out.println((byte)pis.read());
        } finally{
            pis.close();
            pos.close();}
    }
}
```


小结

- 在Java中有数据传输的地方都用到I/O流(通常是文件,网络,内存和标准输入输出等)
- Java 的 IO 流主要包括输入、输出两种 IO 流,每种输入、输出流有可分为字节流和字符流两大类:
 - 字节流以字节为单位来处理输入、输出操作
 - 字符流以字符为单位来处理输入、输出操作
- 众多的流对象协同工作



小结

1. 读写的时候字节流是按字节读写，字符流按字符读写。
2. 字节流适合所有类型文件的数据传输，因为计算机字节（Byte）是电脑中表示信息含义的最小单位。字符流只能够处理纯文本数据，其他类型数据不行，但是字符流处理文本要比字节流处理文本要方便。
3. 在读写文件需要对内容按行处理，比如比较特定字符，处理某一行数据的时候一般会选择字符流。
4. 只是读写文件，和文件内容无关时，一般选择字节流。

小结

- File, File(Input/Output)Stream, RandomAccessFile是处理本地文件的类。
- Data(Input/Output)Stream是一个过滤流的子类,借此可以读写各种基本数据,在文件和网络中经常使用.如: readByte, writeBoolean等。
- Buffered(Input/Output)Stream的作用是在数据送到目的之前先缓存,达到一定数量时再送到目的,以提高程序的运行效率。
- Piped(Input/Output)Stream适合于一个处理的输出作为另一个处理的输入的情况。